

C# - PHẦN 9.1

DELEGATE CƠ BẢN

Tài liệu học - giải thích - ví dụ - bài tập thực hành

Mục tiêu bài học: Hiểu Delegate là gì, cách khai báo, cách gán method, quy tắc khớp chữ ký, cách truyền method vào method khác và chuẩn bị nền tảng cho Action/Func/Predicate, Lambda và LINQ.

Kiến thức nền: Methods, Parameters, Return type, Generic Method, Collections

1. Ý tưởng cốt lõi của Delegate

Sau khi hoàn thành Collections & Generics, ta chuyển sang một khái niệm mới. Delegate ban đầu hơi lạ vì thay vì lưu int, string, Student..., biến Delegate lại có thể tham chiếu đến một method.

Ý quan trọng nhất: Delegate là một kiểu dữ liệu dùng để tham chiếu đến method có chữ ký phù hợp.

2. Bắt đầu bằng một method bình thường

```
public static void SayHello()
{
    Console.WriteLine("Hello!");
}
```

Bình thường ta gọi trực tiếp:

```
SayHello();
```

Kết quả:

```
Hello!
```

```
Program
  ↓
SayHello()
  ↓
Hello!
```

Delegate cho phép ta có thêm một biến đứng giữa:

```
Program
  ↓
biến Delegate
  ↓
SayHello()
  ↓
Hello!
```

3. Khai báo Delegate

```
public delegate void MyDelegate();
```

Đọc là: khai báo một kiểu Delegate tên MyDelegate, có thể tham chiếu đến những method không nhận tham số và không trả về giá trị.

void MyDelegate() trông khá giống khai báo method, nhưng có từ khóa delegate nên đây không phải method. Đây là khai báo một kiểu dữ liệu mới.

```
public class Student
{
}
```

Đoạn trên khai báo kiểu Student. Tương tự:

```
public delegate void MyDelegate();
```

khai báo kiểu MyDelegate.

4. Tạo biến Delegate

```
public delegate void MyDelegate();

public static void SayHello()
{
    Console.WriteLine("Hello!");
}

MyDelegate action = SayHello;
action();
```

action
↓
tham chiếu đến
↓
SayHello()

Khi gọi action(), method SayHello() sẽ được thực thi.

5. Điểm rất quan trọng: không có () khi gán method

Đúng:

```
MyDelegate action = SayHello;
```

Sai:

```
MyDelegate action = SayHello();
```

SayHello	→ method
SayHello()	→ gọi method

Delegate cần tham chiếu đến method, không phải kết quả của việc gọi method.

6. Delegate phải khớp chữ ký method

```
public delegate void MyDelegate();
```

Delegate trên yêu cầu method có return type là void và không có parameter.

```
public static void SayHello()
{
}

public static void SayGoodbye()
{
}
```

Cả hai method đều phù hợp nên có thể gán:

```
MyDelegate action1 = SayHello;
MyDelegate action2 = SayGoodbye;
```

Method không phù hợp:

```
public static void SayHello(string name)
{
    Console.WriteLine($"Hello {name}");
}
```

Delegate yêu cầu: ()
Method thực tế: (string name)
 → Không khớp

7. Delegate có parameter

```
public delegate void GreetingDelegate(string name);
```

Đọc là: Delegate này chỉ tham chiếu được đến những method nhận một string và trả về void.

```
public static void SayHello(string name)
{
    Console.WriteLine($"Hello {name}");
}
```

```
GreetingDelegate greeting = SayHello;
greeting("Hung");
```

greeting("Hung")
 ↓
SayHello("Hung")

8. Một Delegate có thể tham chiếu tới nhiều method cùng chữ ký

```
public static void SayHello(string name)
{
    Console.WriteLine($"Hello {name}");
}

public static void SayGoodbye(string name)
{
    Console.WriteLine($"Goodbye {name}");
}

public delegate void GreetingDelegate(string name);

GreetingDelegate greeting;

greeting = SayHello;
greeting("Hung");

greeting = SayGoodbye;
greeting("Hung");
```

Kết quả: Hello Hung, sau đó Goodbye Hung.

Ghi nhớ: Kiểu Delegate không quan tâm method tên gì. Nó quan tâm chữ ký của method có khớp hay không.

9. Delegate có giá trị trả về

```
public delegate int CalculateDelegate(int a, int b);
```

C# - Phần 9.1: Delegate cơ bản

Delegate này nhận hai int và trả về int.

```
public static int Add(int a, int b)
{
    return a + b;
}

CalculateDelegate calculate = Add;
int result = calculate(5, 3);
Console.WriteLine(result);
```

calculate(5, 3)
↓
Add(5, 3)
↓
8

10. Có thể thay method mà Delegate đang tham chiếu

```
public static int Add(int a, int b)
{
    return a + b;
}

public static int Subtract(int a, int b)
{
    return a - b;
}

public delegate int CalculateDelegate(int a, int b);

CalculateDelegate calculate = Add;
Console.WriteLine(calculate(10, 5)); // 15

calculate = Subtract;
Console.WriteLine(calculate(10, 5)); // 5
```

Cùng một biến calculate có thể tham chiếu đến Add hoặc Subtract miễn là các method cùng chữ ký với Delegate.

11. Tại sao Delegate hữu ích?

```
public static int Calculate(
    int a,
    int b,
    CalculateDelegate operation)
{
    return operation(a, b);
}
```

Method Calculate() không cần biết cụ thể phải cộng hay trừ. Ta truyền hành vi vào thông qua Delegate:

```
int result1 = Calculate(10, 5, Add);
int result2 = Calculate(10, 5, Subtract);
```

```

Calculate(10, 5, Add)
    ↓
    operation = Add
    ↓
    operation(10, 5)
    ↓
    Add(10, 5)
    ↓
    15

Calculate(10, 5, Subtract)
    ↓
    operation = Subtract
    ↓
    operation(10, 5)
    ↓
    Subtract(10, 5)
    ↓
    5

```

Sức mạnh của Delegate: Có thể truyền một method vào một method khác.

12. Liên hệ với kiến thức Generic

```
public static void DisplayValue<T>(T value)
```

Ở Generic, value có thể là int, string, Product, Student... Với Delegate, parameter có thể đại diện cho hành vi.

```

public static int Calculate(
    int a,
    int b,
    CalculateDelegate operation)

```

operation có thể là Add, Subtract, Multiply, Divide... miễn là các method cùng chữ ký.

Generic
→ tổng quát hóa kiểu dữ liệu

Delegate
→ tổng quát hóa hành vi / method

13. Method truyền vào phải khớp chữ ký

```

public delegate decimal SalaryCalculator(
    decimal baseSalary,
    decimal bonus);

```

Method hợp lệ:

```

public static decimal CalculateSalary(
    decimal baseSalary,
    decimal bonus)
{
    return baseSalary + bonus;
}

```

```
Delegate: decimal (decimal, decimal)
Method:   decimal (decimal, decimal)
         → Khớp
```

Hai ví dụ không khớp:

```
public static int CalculateSalary(
    decimal baseSalary,
    decimal bonus)

public static decimal CalculateSalary(
    decimal baseSalary)
```

Ví dụ đầu sai kiểu trả về; ví dụ sau sai số lượng parameter.

14. Delegate là một type

```
public delegate int CalculateDelegate(
    int a,
    int b);
```

Dòng trên tạo ra một kiểu dữ liệu tên CalculateDelegate. Vì vậy ta có thể khai báo biến:

```
CalculateDelegate operation;
```

int	→ chứa số
string	→ chứa chuỗi
Student	→ tham chiếu object Student
CalculateDelegate	→ tham chiếu method phù hợp

15. Cách đọc một Delegate

```
public delegate bool CheckNumber(int number);
```

Đọc là: CheckNumber là một kiểu Delegate có thể tham chiếu tới method nhận một int và trả về bool.

```
public static bool IsPositive(int number)
{
    return number > 0;
}

CheckNumber check = IsPositive;
bool result = check(10);
```

Kết quả là True.

16. Ví dụ gần với Collections

```
public delegate bool NumberCondition(int number);

public static bool IsEven(int number)
{
    return number % 2 == 0;
}
```

```

public static void DisplayNumbers(
    List<int> numbers,
    NumberCondition condition)
{
    foreach (int number in numbers)
    {
        if (condition(number))
        {
            Console.WriteLine(number);
        }
    }
}

List<int> numbers = new List<int>
{
    1, 2, 3, 4, 5, 6
};

DisplayNumbers(numbers, IsEven);

```

Kết quả: 2, 4, 6.

Liên hệ bài sau: Ý tưởng truyền điều kiện vào method này sẽ dẫn trực tiếp đến Lambda và các method như Where(...) trong LINQ.

17. Phân tích ví dụ Collections

Delegate NumberCondition yêu cầu một method nhận int và trả bool. IsEven(int) khớp chữ ký đó.

```
DisplayNumbers(numbers, IsEven);
```

Ở đây method IsEven được truyền vào parameter condition. Sau đó:

```
condition(number)
```

thực chất sẽ gọi method IsEven(number).

```

number = 4
condition(4)
  ↓
IsEven(4)
  ↓
true
  ↓
Console.WriteLine(4)

```

18. Tóm tắt Phần 9.1

1. Delegate là một kiểu dữ liệu tham chiếu tới method.
2. Method được gán cho Delegate phải có chữ ký phù hợp: parameter và kiểu trả về phải tương thích.
3. Khi gán method cho Delegate, viết tên method không có dấu ngoặc ().
4. Gọi Delegate giống như gọi method, ví dụ operation(10, 5).
5. Delegate cho phép truyền hành vi/method vào một method khác.

Câu quan trọng nhất: Generic giúp tổng quát hóa kiểu dữ liệu; Delegate giúp tổng quát hóa hành vi.

19. Bài tập 9.1 - Delegate cơ bản

Khai báo Delegate:

```
public delegate decimal CalculateDelegate(  
    decimal first,  
    decimal second);
```

Viết bốn method sau:

```
public static decimal Add(  
    decimal first,  
    decimal second)  
  
public static decimal Subtract(  
    decimal first,  
    decimal second)  
  
public static decimal Multiply(  
    decimal first,  
    decimal second)  
  
public static decimal Divide(  
    decimal first,  
    decimal second)
```

Divide() phải kiểm tra second == 0 và ném DivideByZeroException.

Sau đó viết:

```
public static decimal Calculate(  
    decimal first,  
    decimal second,  
    CalculateDelegate operation)
```

Method Calculate() không được tự viết phép cộng/trừ/nhân/chia mà phải gọi Delegate:

```
return operation(first, second);
```

Trong Program, kiểm tra lần lượt:

- 10 + 5
- 10 - 5
- 10 * 5
- 10 / 5
- 10 / 0

Thử cả hai cách sử dụng Delegate:

```
CalculateDelegate operation = Add;  
  
decimal result = operation(10, 5);  
  
decimal result = Calculate(10, 5, Add);
```

20. Câu hỏi tự kiểm tra sau bài tập

6. CalculateDelegate là gì?
7. Vì sao Add không có () khi gán?
8. Vì sao Add có thể truyền vào Calculate()?
9. operation(first, second) thực chất đang làm gì?
10. Generic Method và Delegate khác nhau ở điểm cốt lõi nào?

21. Thứ tự thực hành đề xuất

Làm phần khai báo CalculateDelegate và bốn method Add/Subtract/Multiply/Divide trước. Sau khi phần đó chạy đúng và bạn hiểu chữ ký Delegate, tiếp tục hoàn thiện Calculate() và Program.

22. Bảng ghi nhớ nhanh

Khái niệm	Ghi nhớ
Delegate	Kiểu dữ liệu tham chiếu đến method phù hợp
Chữ ký	Kiểu trả về + danh sách parameter phải tương thích
Gán method	MyDelegate action = SayHello;
Gọi Delegate	action(); hoặc operation(a, b);
Không dùng () khi gán	SayHello là method; SayHello() là gọi method
Truyền hành vi	Có thể truyền Delegate vào parameter của method khác
Generic	Tổng quát hóa kiểu dữ liệu
Delegate	Tổng quát hóa hành vi / method
Nền tảng tiếp theo	Action, Func, Predicate → Lambda → LINQ